

EXERCISE-4

Q1)

```
JS q1.js > ...
1  const obj = {
2    a: "foo",
3    b: function () {
4      console.log(this.a);
5    },
6  };
7
8  const c = obj.b;
9
10 obj.b();
11 c();
12
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise4/q1.js"
foo
undefined
```

Explanation:

obj.b() prints foo because we are calling it through a method call, when we call a function as object.method(), this refers to that object. In this case a: "foo" this now points to this, hence printing foo

c() is undefined because in const c = obj.b, we are just passing the reference of obj.b. We called c() as a normal function rather than a method call so this becomes the global object.

To solve this problem we need to bind it using bind().

Q2)

```
JS q2.js
1 console.log([... 'Lydia']);
2
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise4/q2.js"
[ 'L', 'y', 'd', 'i', 'a' ]
```

Explanation: Here we use the spread operator on a string , this operator expands each character of the string into individual elements. Since strings are iterable in javascript this can happen.

Q3)

```
JS q3.js > ...
1 let user={ name:"Piyush", age:24 }
2
3 const {name}= user;
4
5 console.log(name);
6
7 const { name: myName } = { name: 'Lydia' };
8
9 console.log(name);
10
11 |
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise4/q3.js"
Piyush
Piyush
```

Explanation: First console.log(name) prints Piyush because above that in const{name}=user , the code is extracting a property from object user and storing it in name (object destructuring).

Second `console.log(name)` prints Piyush as well because in the second destructuring renames the property to myName, so it doesn't affect the name variable. If we write `console.log(myName)` it will print 'Lydia'.

Q4)

```
JS q4.js > ...
1 console.log({a:1} == {a:1});
2 console.log({a:1} === {a:1});
3
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise4/q4.js"
false
false
```

Explanation: Both return false because objects are compared by reference and not by value.

Everytime `{a:1}` creates a new object in the memory, i.e even being similar each created object is stored at a different location.

==(loose equality)

For objects, `==` compares same objects in memory rather than comparing values. Since the references in this code is different, it prints false.

===(strict equality)

It compares objects by reference and type as well.

Since the references in this code is different as well, it prints false.

Q5)

```
JS q5.js > ...
1  let person = { name: 'Lydia' };
2  const members = [person];
3  person = null;
4
5  console.log(members);
6  |
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise4/q5.js"
[ { name: 'Lydia' } ]
```

Explanation:

The code prints [{name: 'Lydia'}] because objects are stored as reference, not by value. During `const members=[person]` the array stores the reference, i.e. the members array stores reference to the same object (`members[0]->{name:'Lydia'}`).

Now when person=null only the variable person changes and not the object itself, what this means is only the variable changes and the members still holds the reference to the object.

Therefore `console.log(members)` prints : [{name:'Lydia'}]